play_poweron_opendoor 구현

source code

play_poweron_opendoor.v

```
module play_poweron_opendoor(
    input clk,
    input reset,
    input sw,
    input [6:0] btn,
    output [1:0] led,
    output buzzer
    );

    localparam L1 = 22'd49_999;
    localparam L2 = 22'd24_999;
    localparam L3 = 22'd16_666;
    localparam L4 = 22'd12_499;
    localparam R1 = 22'd191_112; // 도 261.6256 Hz
    localparam R2 = 22'd151_685; // 미 329.6276 Hz
    localparam R3 = 22'd127_552; // 솔 391.9954 Hz
    localparam R4 = 22'd90_252;

    reg [22:0] r_cnt;
    reg [6:0] r_buzzer_frequency;
    reg [21:0] r_clk_cnt[6:0]; // 2차원 array
    reg [2:0] r_step_l;
    reg [2:0] r_step_r;
    reg [1:0]r_btn_prev;
    reg r_play;
    reg r_mod;
    integer i;

    wire [21:0] selected_freq_l;
    wire [21:0] selected_freq_r;

    // L 네 단계
    assign selected_freq_l = (r_step_l == 2'd0) ? L1 :
                             (r_step_l == 2'd1) ? L2 :
                             (r_step_l == 2'd2) ? L3 : L4;
```

```
 // R 네 단계
 assign selected_freq_r = (r_step_r == 2'd0) ? R1 :
                          (r_step_r == 2'd1) ? R2 :
                          (r_step_r == 2'd2) ? R3 : R4;

always @(posedge clk or posedge reset) begin
 if (reset) begin
     for (i = 0; i < 7; i = i + 1) begin
         r_clk_cnt[i] <= 22'd0;
         r_buzzer_frequency[i] <= 1'b0;
     end
     r_cnt <= 23'd0;
     r_step_l <= 1'b0;
     r_step_r <= 1'b0;
     r_btn_prev <= 2'b00;
     r_play <= 1'b0;
 end else begin
     r_btn_prev <= btn[1];

     if (sw == 1'b1) begin
         if (!r_play) begin
             // 버튼이 0 → 1로 바뀐 순간 재생 시작
             if (btn[1] && !r_btn_prev[0]) begin
                 r_play <= 1'b1;
                 r_mod <= 1'b0;
                 r_step_l <= 1'b0;
                 r_cnt <= 23'd0;
                 r_clk_cnt[1] <= 22'd0;
                 r_buzzer_frequency[1] <= 1'b0;
             end
             else if (btn[3] && !r_btn_prev[1]) begin
                 r_play <= 1'b1;
                 r_mod <= 1'b1;
                 r_step_r <= 1'b0;
                 r_cnt <= 23'd0;
                 r_clk_cnt[1] <= 22'd0;
                 r_buzzer_frequency[1] <= 1'b0;
             end
         end else begin
```

```
        // 재생 중에는 버튼을 놓아도 계속 진행
    if(r_mod == 1'b0) begin
        if (r_cnt >= 23'd6_999_999) begin
            r_cnt <= 23'd0;
            r_clk_cnt[1] <= 22'd0;
            r_buzzer_frequency[1] <= 1'b0;

            if (r_step_l == 2'd3) begin
                r_play <= 1'b0;       // 네 번째 음까지 재생 완료
                r_step_l <= 2'd0;
            end else begin
                r_step_l <= r_step_l + 2'd1;
            end
        end else begin
            r_cnt <= r_cnt + 23'd1; // 모든 단계에서 시간 계산

            if (r_clk_cnt[1] >= selected_freq_l) begin
                r_clk_cnt[1] <= 22'd0;
                r_buzzer_frequency[1] <= ~r_buzzer_frequency[1];
            end else begin
                r_clk_cnt[1] <= r_clk_cnt[1] + 22'd1;
            end
        end
    end
    else if(r_mod == 1'b1) begin
        if (r_cnt >= 23'd6_999_999) begin
            r_cnt <= 23'd0;
            r_clk_cnt[1] <= 22'd0;
            r_buzzer_frequency[1] <= 1'b0;

            if (r_step_r == 2'd3) begin
                r_play <= 1'b0;       // 네 번째 음까지 재생 완료
                r_step_r <= 2'd0;
            end else begin
                r_step_r <= r_step_r + 2'd1;
            end
        end else begin
            r_cnt <= r_cnt + 23'd1; // 모든 단계에서 시간 계산
```

```
                    if (r_clk_cnt[1] >= selected_freq_r) begin
                        r_clk_cnt[1] <= 22'd0;
                        r_buzzer_frequency[1] <= ~r_buzzer_frequency[1];
                    end else begin
                        r_clk_cnt[1] <= r_clk_cnt[1] + 22'd1;
                    end
                end
            end
            end
        end
        else begin
            r_play <= 1'b0;
            r_step_l <= 1'b0;
            r_step_r <= 1'b0;
            r_cnt <= 23'd0;
            r_clk_cnt[1] <= 22'd0;
            r_buzzer_frequency[1] <= 1'b0;
        end
    end
    end

    assign buzzer = |r_buzzer_frequency;
endmodule
```

top_buzzer.v

```
`timescale 1ns / 1ps

module top_buzzer(
    input clk,
    input reset,
    input [6:0] btn,
    input sw,
    output [1:0] led,
    output buzzer
    );

    wire [6:0] w_btn;
    wire buzzer_melody;
    wire buzzer_poweron;
```

```
    btn_debouncer u_btn_debouncer(
        .clk(clk),
        .reset(reset),
        .noisy_btn(btn),
        .clean_btn(w_btn)
    );

    play_melody u_play_melody(
        .clk(clk),
        .reset(reset),
        .btn(w_btn),
        .led(led),
        .buzzer(buzzer_melody),
        .sw(sw)
    );

    play_poweron_opendoor u_play_poweron_opendoor(
        .clk(clk),
        .reset(reset),
        .btn(w_btn),
        .led(led),
        .buzzer(buzzer_poweron),
        .sw(sw)
    );

    assign buzzer = (sw == 0) ? buzzer_melody : buzzer_poweron;
endmodule
```

tb_buzzer.v

```
`timescale 1ns / 1ps

module tb_buzzer;

    reg clk = 0;
    reg reset = 1;
    reg [6:0] btn = 7'b1100000; // 외부 풀업 버튼 5, 6은 기본 1
    reg sw = 0;
```

```
    wire [1:0] led;
    wire buzzer;

    top_buzzer dut (
        .clk(clk),
        .reset(reset),
        .btn(btn),
        .sw(sw),
        .led(led),
        .buzzer(buzzer)
    );

    always #5 clk = ~clk; // 100 MHz

    initial begin
        #100;
        reset = 0;

        // sw=0: 일반 음계, 왼쪽 버튼 100 ms 누르기
        #100;
        btn[1] = 1;
        #100_000_000;
        btn[1] = 0;
        #100_000_000;

        // sw=1: 왼쪽 버튼 한 번 누르고 멜로디 완료 대기
        sw = 1;
        #100;
        btn[1] = 1;
        #100_000_000;
        btn[1] = 0;
        #300_000_000;

        // sw=0: 일반 음계로 다시 돌아오는지 확인
        sw = 0;
        #100;
        btn[1] = 1;
        #100_000_000;
        btn[1] = 0;
```

```
        #100_000_000;

        $finish;
    end

endmodule
```

테스트 벤치 결과

